FACULTY OF ENGINEERING SCIENCES AND TECHNOLOGY

HAMDARD UNIVERSITY KARACHI

Department of Information Technology (DIT)

Infrastructure Automation

Lecture Notes and Weekly Assessment Questions (Weeks 1–15)

Course Code	CCIA-5214
Course Level	Undergraduate
Credit Hours	2 + 1 (Theory + Lab)
Semester / Term	Semester 6 – Fall 2026
Degree Program	Cloud Application Development and Operations
Instructor	Iqbal Uddin Khan
Email	iqbaluddin.khan@hamdard.edu.pk

Open-source policy
All hands-on activities and tool recommendations in these notes rely exclusively on open-source or free-tier software so that every exercise can be reproduced without licensing cost.

Table of Contents

Table of Contents **2**
A Note on These Lecture Notes **3**
Week 1: Introduction to Infrastructure Automation and its Role in Cloud Computing **4**
Hour 1 - Foundations of Infrastructure Automation 4
Hour 2 - The Role of Automation in Cloud Operations 4
Hour 3 - Lab: Setting Up the Automation Toolchain 5
AI-Restricted Assessment Questions (10) 5
Week 2: Principles of Infrastructure as Code (IaC) **7**
Hour 1 - Core Concepts of IaC 7
Hour 2 - IaC Lifecycle and Benefits 7
Hour 3 - Lab: First Steps with Terraform/OpenTofu 7
AI-Restricted Assessment Questions (10) 8
Week 3: Scripting Basics for Automation (Python, Bash, PowerShell) **10**
Hour 1 - Bash Scripting Fundamentals 10
Hour 2 - Python for Infrastructure Automation 10
Hour 3 - Lab: Writing Automation Scripts 10
AI-Restricted Assessment Questions (10) 11
Week 4: Configuration Management Tools Overview (Ansible, Puppet, Chef) **13**
Hour 1 - Configuration Management Concepts 13
Hour 2 - Ansible Architecture, and Puppet/Chef Comparison 13
Hour 3 - Lab: Configuring a Server with Ansible 13
AI-Restricted Assessment Questions (10) 14
Week 5: Orchestration Frameworks and Automated Workflows **16**
Hour 1 - Containers and the Case for Orchestration 16
Hour 2 - Kubernetes Architecture 16
Hour 3 - Lab: Deploying and Scaling with Kubernetes 16
AI-Restricted Assessment Questions (10) 17
Week 6: Terraform Deep Dive - Provisioning Cloud Resources **19**
Hour 1 - Terraform Core Concepts Revisited in Depth 19
Hour 2 - Modules, Workspaces, and Remote State 19
Hour 3 - Lab: Provisioning and Destroying Resources 19
AI-Restricted Assessment Questions (10) 21
Week 7: CloudFormation Concepts and Multi-Cloud Automation **22**
Hour 1 - Multi-Cloud Automation Concepts 22
Hour 2 - Conceptual Comparison of Approaches 22
Hour 3 - Assignment: Comparative Analysis Exercise 23
AI-Restricted Assessment Questions (10) 23
Week 8: Midterm Review and Assessment **25**
Hour 1 - Conceptual Review (CLO-1 Focus) 25
Hour 2 - Applied/Hands-On Review (CLO-2 Focus) 25
Hour 3 - Midterm Examination (Theory + Lab) 25
AI-Restricted Assessment Questions (10) 26
Week 9: Automated Provisioning and Scaling **27**
Hour 1 - Scaling Concepts and Triggers 27

Hour 2 - Autoscaling in Kubernetes 27
Hour 3 - Lab: Configuring the Horizontal Pod Autoscaler 27
AI-Restricted Assessment Questions (10) 28
Week 10: Monitoring and Logging in Automated Infrastructure 30
Hour 1 - Observability Fundamentals 30
Hour 2 - Prometheus and Grafana 30
Hour 3 - Lab: Deploying a Monitoring Stack 30
AI-Restricted Assessment Questions (10) 31
Week 11: Integration with CI/CD Pipelines 33
Hour 1 - CI/CD Concepts 33
Hour 2 - Jenkins and GitHub Actions 33
Hour 3 - Lab: Building an Automation Pipeline 33
AI-Restricted Assessment Questions (10) 34
Week 12: Security and Compliance in Automation 36
Hour 1 - Security Principles in Automation 36
Hour 2 - Open-Source Compliance and Scanning Tools 36
Hour 3 - Lab: Auditing and Scanning 36
AI-Restricted Assessment Questions (10) 37
Week 13: Enterprise Automation Case Studies 39
Hour 1 - Case Study Methodology and Presentation 39
Hour 2 - Group Analysis Workshop 39
Hour 3 - Group Presentations and Discussion 39
AI-Restricted Assessment Questions (10) 40
Week 14: Best Practices in Infrastructure Automation 41
Hour 1 - IaC and Code-Quality Best Practices 41
Hour 2 - GitOps and Organizational Best Practices 41
Hour 3 - Assignment: Writing Automation Best-Practice Guidelines 42
AI-Restricted Assessment Questions (10) 42
Week 15: Capstone Lab - Design an Infrastructure as Code Solution 44
Hour 1 - Capstone Briefing and Requirements 44
Hour 2 - Architecture Design Session 44
Hour 3 - Guided Implementation and Consultation 45
AI-Restricted Assessment Questions (10) 46

A Note on These Lecture Notes

These notes expand each week of the approved course outline into a three-hour lecture session (aligned with the 2+1 credit-hour structure: two theory hours and one lab/practical hour), each explicitly mapped to its Course Learning Outcome (CLO). Every tool referenced for hands-on activities is open-source or offered on a free tier, so that students can reproduce every exercise on a personal laptop without incurring licensing cost.

Each week concludes with ten assessment questions labeled as AI-Restricted Questions. These are deliberately written as scenario-based, applied, and reflective prompts that require the student to reference their own specific lab output, their own configuration files, or their own reasoning about a contextual trade-off. Because a correct answer depends on details unique to the student's own hands-on session and critical judgment rather than on retrievable factual recall, these questions are intended to resist being answered adequately through a generic, unattributed query to an AI system, and instead require genuine engagement with the week's practical work.

> **Tooling note (Fall 2026 edition)**
> Terraform releases from version 1.6 onward are distributed under HashiCorp's Business Source License, which is source-available but not open source. To honor the course's open-source tooling policy, these notes and the Lab Manual use **OpenTofu** (the MPL-2.0 community fork, command `tofu`). Configuration syntax and the init, plan, apply, destroy workflow are identical, so students who have Terraform installed may substitute `terraform` for `tofu` wherever it appears.
>
> Puppet's open-source distribution is now maintained by the community as **OpenVox**; the `puppet` command-line interface and manifest language are unchanged (see Lab 4 in the Lab Manual).

Week 1: Introduction to Infrastructure Automation and its Role in Cloud Computing

Mapped CLO: CLO-1

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Define infrastructure automation and distinguish it from traditional manual system administration.
- Explain the evolution of infrastructure management from bare-metal to cloud-native environments.
- Identify the categories of automation: provisioning, configuration management, and orchestration.
- Describe how automation supports scalability, reliability, and operational efficiency in cloud computing.

Hour 1 - Foundations of Infrastructure Automation

Begin with a working definition of infrastructure automation: the use of scripts, tools, and declarative configurations to provision, configure, and manage computing resources with minimal human intervention.

Trace the historical progression: manual server racking and cabling, shell-script based administration, the rise of virtualization, and the shift to cloud and container-based infrastructure.

Discuss the pain points of manual operations: configuration drift, human error, slow provisioning times, inconsistent environments between development, testing, and production, and difficulty in scaling teams alongside infrastructure.

Introduce core terminology that will recur throughout the course: Infrastructure as Code (IaC), Configuration Management (CM), Orchestration, Provisioning, Idempotency, and Declarative versus Imperative approaches.

Use a simple narrative example: a team manually configuring ten web servers versus a team using a script or IaC tool to configure the same ten servers, highlighting time, consistency, and error-rate differences.

Hour 2 - The Role of Automation in Cloud Operations

Explain how cloud computing characteristics - on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service - depend on automation to be realized in practice.

Discuss the automation maturity model: ad-hoc scripts, standardized scripts, configuration management, full Infrastructure as Code, and self-healing/auto-remediating infrastructure.

Cover how automation enables elasticity (auto-scaling), reliability (self-healing, automated failover), and cost-efficiency (automated shutdown of idle resources, right-sizing).

Introduce the DevOps culture connection: automation as the technical enabler of continuous integration and continuous delivery (CI/CD), and the shared responsibility model between development and operations teams.

Discuss automation and organizational risk: benefits of consistency and speed versus new risks such as automated propagation of misconfiguration, and the resulting need for testing, version control, and review of automation code.

Hour 3 - Lab: Setting Up the Automation Toolchain

Guide students through installing and configuring the free, open-source tools that will be used throughout the semester: Git, a text editor/IDE (VS Code), and a terminal/shell environment (Bash on Linux/macOS or WSL on Windows).

Create a GitHub account (free tier) and initialize a personal course repository to store all weekly lab work and assignments.

Write and execute a first automation script in Bash that performs a repetitive administrative task (e.g., creating a folder structure, generating log files with timestamps, or checking free disk space) to concretely illustrate the difference between a manual task and an automated one.

Commit and push the first script to the student's GitHub repository, establishing the version-control habit that will be reinforced every week.

```
#!/bin/bash
# Example: simple automation script (Bash)
echo "Automated Environment Setup - $(date)"
mkdir -p ~/infra-automation-course/week1
cd ~/infra-automation-course/week1
df -h > disk_report_$(date +%F).txt
echo "Report generated successfully."
```

Recommended Open-Source / Free Tools

- Git (open source)
- Visual Studio Code (free)
- Bash / WSL (free)
- GitHub free tier

AI-Restricted Assessment Questions (10)

1. Your organization currently provisions new servers manually, taking three days per server with an error rate of 15%. Propose an automation strategy (naming the categories of automation involved) and justify how each category reduces the specific problems described.
2. A colleague argues that 'automation is only useful for large enterprises with hundreds of servers.' Using the automation maturity model, construct a counter-argument for a 10-person start-up.
3. Explain, using your own words and a concrete example from your own device or a lab you have configured, the difference between idempotent and non-idempotent operations.

4. Design (in prose, not code) a decision framework a systems administrator could use to decide whether a task should be automated with a shell script, a configuration management tool, or an orchestration framework.
5. Critically evaluate: 'Automation eliminates human error in infrastructure management.' Identify at least two ways automation can introduce or amplify errors rather than remove them.
6. Using the disk-report script created in the lab, describe two modifications you would make to turn it into a reusable, parameterized automation tool rather than a one-off script.
7. A start-up wants rapid elasticity but has a strict monthly cloud budget. Explain how automated scaling and automated de-provisioning would both help and where they could conflict with the budget constraint.
8. Compare, based on today's lab experience, the time and cognitive effort of writing one Bash script for a repetitive task versus performing that task manually five times. Support your answer with specific observations from your own session.
9. Explain why version control (Git) is treated as a foundational tool for infrastructure automation rather than only for application source code.
10. Propose one real risk that automated infrastructure introduces into your own future project or job role, and one concrete safeguard (process or tool) you would put in place to mitigate it.

Week 2: Principles of Infrastructure as Code (IaC)

Mapped CLO: CLO-1

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Define Infrastructure as Code and differentiate declarative and imperative approaches.
- Explain idempotency, version control, and repeatability as core IaC principles.
- Describe the IaC lifecycle: write, plan, apply, and destroy.
- Identify the benefits of IaC for disaster recovery and environment consistency.

Hour 1 - Core Concepts of IaC

Define Infrastructure as Code as the practice of managing and provisioning infrastructure through machine-readable definition files rather than manual configuration or interactive tools.

Contrast declarative IaC (describe the desired end-state; the tool determines the steps - e.g., Terraform/OpenTofu) with imperative IaC (describe the exact steps to reach a state - e.g., a sequence of shell commands or Bash scripts).

Explain idempotency in the IaC context: running the same configuration multiple times produces the same result without unintended side effects, and why this property is essential for safe automation at scale.

Discuss why infrastructure definitions should be stored in version control alongside application code, enabling code review, history tracking, rollback, and collaboration.

Hour 2 - IaC Lifecycle and Benefits

Walk through the standard IaC life-cycle: Write (author configuration files), Plan (preview the changes that will be made), Apply (execute the changes against real infrastructure), and Destroy (tear down resources when no longer needed).

Discuss configuration drift - the gradual divergence between the documented/intended state and the actual state of infrastructure - and how IaC tools detect and correct drift.

Explain the disaster-recovery benefit of IaC: infrastructure can be recreated from code in a new region or account within minutes, rather than depending on manual run-books or institutional memory.

Discuss environment consistency: using the same IaC definitions (parameterized) for development, staging, and production to eliminate 'it works on my machine' class problems.

Hour 3 - Lab: First Steps with Terraform/OpenTofu

Install OpenTofu (the open-source, community-governed fork of Terraform) or Terraform Community Edition on the student lab machines.

Initialize a first project directory and write a minimal configuration using the local provider (creating local files/directories) so that no cloud account or cost is required.

Walk through 'init', 'plan', and 'apply', observing the plan output and confirming that a second 'apply' with no changes reports no differences - demonstrating idempotency directly.

Modify the configuration and re-run 'plan' to observe how the tool calculates only the incremental change (drift correction), then run 'destroy' to clean up.

```
# main.tf - minimal declarative example (local provider, no cloud
cost)
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

resource "local_file" "course_note" {
  content  = "Week 2 - Infrastructure as Code Lab"
  filename = "${path.module}/week2_output.txt"
}

# Commands
# tofu init
# tofu plan
# tofu apply
# tofu destroy
```

Recommended Open-Source / Free Tools

- OpenTofu / Terraform (open source)
- Git
- VS Code with HCL/Terraform extension (free)

AI-Restricted Assessment Questions (10)

1. A junior engineer writes a Bash script that runs 'useradd' for a new employee every time it is executed, failing with an error on the second run if the user already exists. Explain why this script is not idempotent and rewrite the logic (in prose) so that it becomes idempotent.
2. Compare declarative and imperative IaC using the local-provider lab exercise you completed today: which category does Terraform/OpenTofu belong to, and what evidence from your own 'terraform plan' output supports your answer?
3. Your production environment has silently diverged from its documented configuration after a manual emergency fix six months ago. Describe, step by step, how an IaC-based workflow would have detected and corrected this drift.
4. Explain, referencing the plan/apply/destroy cycle from your lab, why 'terraform plan' is considered a safety mechanism rather than merely an informational step.
5. A company stores its Terraform/OpenTofu configuration files only on a single engineer's laptop, not in Git. Identify three concrete risks this creates and propose a remediation for each.

6. During today's lab, re-run 'apply' a second time without changing the configuration. Report and interpret the exact output you observed, connecting it explicitly to the principle of idempotency.
7. Design (in prose) a disaster-recovery scenario for a small e-commerce website using IaC principles: what would need to be stored, and what commands or lifecycle stages would be executed to restore service in a new environment within one hour?
8. Critically assess this claim: 'Because IaC is code, it eliminates the need for testing infrastructure changes.' Provide a specific counter-example.
9. Explain why using the exact same IaC module (with different variable values) for staging and production is preferable to hand-writing separate configurations for each environment, using a concrete parameter example (e.g., instance count or file path) from your own lab file.
10. Reflect on your own lab session: identify one place where you had to correct an error in your .tf file. Explain what the plan output told you, and how that feedback loop compares to a purely manual configuration approach.

Week 3: Scripting Basics for Automation (Python, Bash, PowerShell)

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Apply Bash scripting constructs (variables, loops, conditionals, functions) to automate system tasks.
- Apply Python scripting fundamentals relevant to infrastructure automation (file handling, subprocess execution, basic parsing).
- Recognize the role of PowerShell in Windows-based infrastructure automation.
- Write and execute simple, reusable automation scripts.

Hour 1 - Bash Scripting Fundamentals

Cover Bash scripting essentials: shebang lines, variables, command substitution, string and arithmetic operations.

Cover control structures: if/elif/else conditionals, for and while loops, and case statements, each illustrated with an infrastructure-relevant example (e.g., looping over a list of hostnames).

Cover functions in Bash: defining reusable blocks of logic, passing arguments, and returning exit codes for use in larger automation pipelines.

Discuss best practices: using 'set -euo pipefail' for safer scripts, quoting variables, and adding comments/documentation for maintainability.

Hour 2 - Python for Infrastructure Automation

Introduce Python as a higher-level scripting language commonly used for infrastructure tooling due to its readability and rich standard library.

Cover the 'os' and 'pathlib' modules for file and directory operations, and the 'subprocess' module for invoking shell commands from within Python.

Cover reading and writing structured configuration data using the 'json' and 'yaml' (PyYAML, open source) libraries, since most automation tools (Ansible, Kubernetes) use YAML.

Demonstrate a short script that reads a list of server names from a text file and iterates over them performing a simulated check (e.g., a ping or a mock HTTP check against a local test endpoint).

Hour 3 - Lab: Writing Automation Scripts

Lab Task A: Write a Bash script that automates local backup of a directory, adding a timestamp to the archive name and logging success/failure to a log file.

Lab Task B: Write a Python script that reads a CSV or text file listing multiple 'servers' (simulated as local folders/ports) and reports free disk space or a simulated health status for each.

Briefly introduce PowerShell Core (open source, cross-platform) syntax equivalents for the Bash concepts covered, so students recognize the parallel constructs (Get-ChildItem, ForEach-Object, If/Else) even without a full hands-on session.

Students commit both scripts to their GitHub repository under a week folder, continuing the version-control habit from Week 1.

```
#!/bin/bash
# Bash: simple backup automation
set -euo pipefail
SRC="$HOME/infra-automation-course"
DEST="$HOME/backups"
mkdir -p "$DEST"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
tar -czf "$DEST/backup_$TIMESTAMP.tar.gz" "$SRC" \
  && echo "Backup succeeded: $TIMESTAMP" >> backup.log \
  || echo "Backup FAILED: $TIMESTAMP" >> backup.log

# Python: disk usage check across a list of servers
import shutil
servers = ["server1", "server2", "server3"]
for s in servers:
    total, used, free = shutil.disk_usage("/")
    print(f"{s}: free={free // (2**30)} GB")
```

Recommended Open-Source / Free Tools

- Bash
- Python 3 (free)
- PowerShell Core (open source)
- VS Code

AI-Restricted Assessment Questions (10)

1. Your backup script from today's lab uses a fixed destination path. Rewrite (in prose, describing the logic) the script so it accepts the source and destination as parameters instead of hard-coded values, and explain why this makes the script more suitable for reuse across projects.
2. Explain, with reference to the 'set -euo pipefail' line used in your lab script, what each of the three options does and give a specific scenario in which omitting one of them would cause a silent failure.
3. Your Python disk-usage script currently checks the same local disk for every 'server' in the list because the servers are simulated. Explain what would need to change in the script's logic to make it check a genuinely remote server, without writing the full remote-connection code.
4. Compare a for-loop in Bash with a for-loop in Python, using the specific server-list examples from today's lab, and identify one situation where Bash looping over text is more error-prone than Python looping over a list object.

5. A script you have written works correctly when run manually but fails when triggered by a scheduled job (e.g., cron) at 3 AM. Propose three specific debugging steps tailored to this scenario.
6. Explain why infrastructure automation teams commonly choose Python over Bash once a script exceeds roughly 100 lines or needs to parse structured data such as JSON or YAML, referencing the tools used in this course.
7. Design (in prose) a function-based refactor of your Bash backup script from Hour 3, describing what each function would be responsible for and why breaking the script into functions improves maintainability.
8. PowerShell, Bash, and Python all support looping and conditionals but differ in typical use cases. Based on today's lecture, justify which language you would choose to automate a task on a mixed Windows/Linux estate, and why.
9. Your log file (backup.log) grows indefinitely over time. Propose a script-level or scheduling-level solution to manage log growth, and justify your choice.
10. Reflecting on your own Hour 3 lab session, describe one bug you encountered (syntax or logic) and the exact steps you took to identify and fix it.

Week 4: Configuration Management Tools Overview (Ansible, Puppet, Chef)

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Define configuration management and distinguish agent-based from agentless architectures.
- Describe the architecture of Ansible: inventory, playbooks, modules, and roles.
- Compare Ansible with Puppet and Chef at a conceptual level.
- Configure a server using an Ansible playbook.

Hour 1 - Configuration Management Concepts

Define configuration management as the practice of systematically handling changes to a system's configuration so that it maintains a known, consistent state over time.

Explain the distinction between agent-based tools (Puppet, Chef - require software installed and running on managed nodes) and agentless tools (Ansible - communicates over SSH/WinRM without a persistent agent).

Introduce YAML syntax fundamentals since it underlies Ansible playbooks and is used broadly across the automation ecosystem covered in this course (Kubernetes, CI/CD pipelines).

Discuss the trade-offs of each architecture: agentless tools are simpler to bootstrap but may not scale as efficiently to very large fleets; agent-based tools offer continuous enforcement but add operational overhead of managing the agent itself.

Hour 2 - Ansible Architecture, and Puppet/Chef Comparison

Detail Ansible's architecture: the control node, the inventory file (listing managed hosts/groups), playbooks (YAML files describing desired tasks), modules (units of work such as 'apt', 'copy', 'service'), and roles (reusable, structured collections of tasks/handlers/variables).

Introduce Puppet's declarative manifest language and its agent/master pull-based model, and Chef's Ruby-based recipes/cookbooks with its similar agent/pull architecture, at a conceptual level using their free/open-source community editions.

Compare the three tools across dimensions relevant to decision-making: learning curve, agent requirement, primary language/syntax, and typical use cases in industry.

Discuss idempotency in the configuration-management context specifically: how Ansible modules check current state before making changes, avoiding repeated or destructive re-application.

Hour 3 - Lab: Configuring a Server with Ansible

Set up a local test environment using Vagrant with VirtualBox (both free/open source) to create one or two lightweight virtual machines that will act as 'managed nodes' for Ansible.

Write an inventory file listing the Vagrant VM(s) and confirm connectivity using 'ansible all -m ping'.

Write a simple playbook that installs a package (e.g., nginx or a lightweight web server), starts and enables the service, and copies a custom configuration/index file to the node.

Re-run the playbook a second time and observe Ansible reporting the relevant tasks as 'ok' rather than 'changed', reinforcing the idempotency concept from Weeks 2 and 3 in a configuration-management context.

```
# inventory.ini
[webservers]
192.168.56.10 ansible_user=vagrant

# playbook.yml
---
- hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
        update_cache: yes
    - name: Ensure nginx is running
      service:
        name: nginx
        state: started
        enabled: yes

# Run: ansible-playbook -i inventory.ini playbook.yml
```

Recommended Open-Source / Free Tools

- Ansible (open source)
- Vagrant (free)
- VirtualBox (open source)
- Git

AI-Restricted Assessment Questions (10)

1. Explain, using the ping-module test from today's lab, why Ansible is described as 'agentless' and identify what mechanism (protocol) it relies on instead of a persistent agent.
2. Your organization manages 5,000 servers with a requirement for continuous, automatic enforcement of configuration even if a server is offline during a scheduled push. Based on the architectural differences discussed in lecture, would Ansible's default push-based, agentless model or an agent-based tool such as Puppet be more appropriate, and why?
3. In today's lab you re-ran the same playbook twice. Report what changed between the 'changed' count on the first run and the second run, and explain the underlying Ansible mechanism responsible for that difference.

4. Rewrite (in prose) the lab playbook's logic to additionally ensure a specific configuration file's content is present, and explain which Ansible module you would use and why it is preferable to using the generic shell/command module for this purpose.
5. Compare Ansible's YAML playbooks with Puppet's manifest language and Chef's Ruby-based recipes in terms of the learning curve for a team of systems administrators with no programming background.
6. A student claims 'Ansible cannot manage Windows machines because it uses SSH.' Evaluate this claim and correct any inaccuracies based on the architecture discussed in lecture.
7. Explain why placing repeated logic (e.g., installing and configuring the same web server across multiple projects) into an Ansible role, rather than duplicating tasks across playbooks, aligns with the IaC principles introduced in Week 2.
8. During the lab, if the 'ansible all -m ping' command had failed with an SSH connection error, list three specific diagnostic steps you would take, referencing the inventory file and Vagrant network configuration.
9. Critically discuss one scenario where an agent-based configuration management tool (Puppet/Chef) would be operationally safer than Ansible's agentless push model, and one scenario where the reverse is true.
10. Reflect on your own playbook.yml file from today: identify one task where using a specific Ansible module (rather than a raw shell command) improved idempotency, and explain how.

Week 5: Orchestration Frameworks and Automated Workflows

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Distinguish orchestration from configuration management and provisioning.
- Describe containerization fundamentals using Docker.
- Describe Kubernetes architecture: pods, deployments, services, and the control plane.
- Deploy and scale a containerized application using an orchestration platform.

Hour 1 - Containers and the Case for Orchestration

Define orchestration as the automated coordination and management of multiple, often interdependent, computing tasks or services (e.g., scheduling containers across a cluster, managing service dependencies, healing failed instances) - distinct from configuration management (which manages the state of individual hosts).

Introduce containerization using Docker: images, containers, Dockerfiles, and the benefit of packaging an application with its dependencies for consistent execution across environments.

Explain why, once an application is split into many containers across many hosts, manual placement and scaling become impractical - motivating the need for an orchestration framework.

Walk through building a simple Docker image from a Dockerfile and running it locally as a preparation step for the orchestration concepts to follow.

Hour 2 - Kubernetes Architecture

Introduce Kubernetes as the leading open-source container-orchestration platform and describe its control plane components: API server, scheduler, controller manager, and etcd (the cluster's data store).

Describe worker-node components: kubelet and the container runtime, and the core Kubernetes objects: Pods (the smallest deployable unit), Deployments (managing replica sets and rolling updates), and Services (stable networking/load-balancing for a set of pods).

Explain declarative management in Kubernetes: applying a YAML manifest describing desired state (e.g., '3 replicas of this image') and letting the control plane continuously reconcile actual state to match it - directly connecting back to the IaC principles from Week 2.

Discuss self-healing: how Kubernetes automatically restarts failed containers and reschedules pods from failed nodes without human intervention.

Hour 3 - Lab: Deploying and Scaling with Kubernetes

Install Minikube or Kind (Kubernetes in Docker) - both free, open-source, single-node Kubernetes distributions suitable for a laptop lab environment.

Write a Deployment manifest for a simple containerized application (e.g., a lightweight open-source web server image) and apply it with 'kubectl apply -f'.

Expose the deployment using a Service manifest and access the application locally through Minikube's tunnelling/port-forward features.

Scale the deployment by changing the 'replicas' field and re-applying, observing how Kubernetes automatically creates or removes pods to match the new desired state.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginx:latest
          ports:
            - containerPort: 80

# Commands
# kubectl apply -f deployment.yaml
# kubectl scale deployment web-app --replicas=4
# kubectl get pods -w
```

Recommended Open-Source / Free Tools

- Docker (free/open source Engine)
- Kubernetes (open source)
- Minikube or Kind (open source)
- kubectl (free)

AI-Restricted Assessment Questions (10)

1. Explain, using your own Week 4 Ansible lab and today's Kubernetes lab as reference points, the practical difference between 'configuration management' and 'orchestration'.
2. During today's lab you scaled the deployment from 2 to 4 replicas. Describe exactly what the Kubernetes control plane components (API server, scheduler, controller manager) each did, in sequence, to fulfil that request.
3. A container in your deployment crashes repeatedly. Based on the self-healing behavior discussed in lecture, predict what Kubernetes will do, and explain the specific control-plane mechanism responsible.

4. Compare running the same application as a single Docker container on one machine versus as a Kubernetes Deployment with 3 replicas across a cluster: identify two specific operational capabilities gained and one added complexity cost.
5. Your deployment.yaml specifies image 'nginx:latest'. Critically evaluate why using a specific version tag instead of ':latest' is considered a best practice for production automation, connecting your answer to the idempotency principle from Week 2.
6. Explain the role of a Kubernetes Service in front of your Deployment from today's lab, and describe what would break if pods were accessed directly by their individual (ephemeral) IP addresses instead.
7. Propose a scenario in your own words where an application would need orchestration (Kubernetes) but would not benefit meaningfully from further configuration-management automation (Ansible/Puppet/Chef), and justify your reasoning.
8. During the lab, if 'kubectl get pods' showed a pod stuck in 'Pending' state, list three specific things you would check, referencing the resources of your local Minikube/Kind cluster.
9. Explain why Kubernetes manifests are considered a form of Infrastructure as Code, linking this explicitly to the declarative-versus-imperative distinction from Week 2.
10. Reflect on your own deployment.yaml: propose one additional field or object (e.g., resource limits, readiness probe) you would add before considering this configuration production-ready, and justify its purpose.

Week 6: Terraform Deep Dive - Provisioning Cloud Resources

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Explain Terraform/OpenTofu core concepts: providers, resources, variables, and state.
- Use modules and workspaces to organize infrastructure code.
- Configure remote state using an open-source backend.
- Provision, modify, and destroy infrastructure using an open-source or simulated provider.

Hour 1 - Terraform Core Concepts Revisited in Depth

Revisit and deepen the Week 2 introduction: providers (plugins that translate HCL into API calls for a specific platform), resources (the individual infrastructure objects being managed), and data sources (read-only references to existing infrastructure).

Explain input variables (parameterize configuration) and output values (exposing information from a configuration for use elsewhere), with concrete syntax examples.

Explain the Terraform state file in detail: why it exists (mapping configuration to real-world resource IDs), the risks of manual state editing, and why state should never be committed with secrets in plain text to public version control.

Discuss the 'terraform graph'/dependency-resolution concept: how Terraform automatically determines the correct order to create or destroy interdependent resources.

Hour 2 - Modules, Workspaces, and Remote State

Introduce modules as reusable, encapsulated groups of resources with their own inputs and outputs, and explain how they support the DRY (Don't Repeat Yourself) principle across environments.

Introduce workspaces as a mechanism for managing multiple distinct states (e.g., dev/staging/prod) from a single configuration directory, and discuss their appropriate and inappropriate use cases.

Discuss remote state back-ends using open-source-friendly options (e.g., a local back-end for the course, or a self-hosted MinIO (open source, S3-compatible) bucket) so that state can be shared across a team without manual file transfer.

Discuss locking: how remote back-ends can prevent two engineers from applying conflicting changes simultaneously.

Hour 3 - Lab: Provisioning and Destroying Resources

Using LocalStack (open-source edition) to simulate cloud resources without cost or an account, or the Docker provider for Terraform/OpenTofu, students write a configuration provisioning multiple related resources (e.g., a storage bucket and a queue, or multiple Docker containers with a defined network).

Parameterize the configuration using input variables (e.g., resource count, naming prefix) and confirm via 'plan' that changing a variable produces the expected incremental change.

Refactor part of the configuration into a simple local module and re-apply, demonstrating no functional change but improved organization.

Run 'terraform destroy' and confirm complete removal of all provisioned resources, reinforcing responsible resource lifecycle management.

```
# variables.tf
variable "container_count" {
  type    = number
  default = 2
}

# versions.tf
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

# main.tf (Docker provider - free/open source, no cloud cost)
resource "docker_image" "nginx" {
  name = "nginx:1.25"
}

resource "docker_container" "web" {
  count = var.container_count
  name  = "web-${count.index}"
  image = docker_image.nginx.image_id
}

# Commands
# tofu init
# tofu plan -var="container_count=3"
# tofu apply -var="container_count=3"
# tofu destroy
```

Recommended Open-Source / Free Tools

- OpenTofu/Terraform (open source)
- Docker provider (free)
- LocalStack Community Edition (open source)
- MinIO (open source, optional)

AI-Restricted Assessment Questions (10)

1. Explain, using your own container_count variable exercise from today's lab, why changing a single variable value and re-running 'plan' is safer than editing the resource block directly for every environment.
2. Your team's Terraform state file for a shared project is currently stored only on one engineer's laptop. Identify two specific failure scenarios this creates and explain how a remote back-end with locking would prevent each.
3. During the lab you refactored part of your configuration into a module. Describe what changed in the plan output (if anything) after the refactor, and explain why a well-executed module refactor should not, by itself, alter real infrastructure.
4. Critically discuss the statement: 'Since Terraform state is just a JSON file, it is safe to edit manually whenever plan shows an unexpected change.' Identify the specific risks of this practice.
5. Explain the purpose of Terraform workspaces and describe one scenario in your own course project where using a workspace per environment (dev/staging/prod) would be appropriate, and one scenario where using entirely separate directories/state files would be safer.
6. Your 'terraform destroy' command in today's lab removed multiple interdependent resources in a specific order. Explain how Terraform determined that order without you specifying it manually.
7. A teammate proposes committing the Terraform state file to the same public GitHub repository as the configuration code. Evaluate the specific risks of this proposal, referencing what a state file typically contains.
8. Compare provisioning the same set of resources (e.g., 3 containers) using a raw Bash script with Docker CLI commands versus using the Terraform/OpenTofu configuration from today's lab. Identify two concrete advantages of the declarative approach for this specific case.
9. Explain why LocalStack or the Docker provider is used in this course instead of a real commercial cloud account, and identify one limitation this substitution introduces for learning purposes.
10. Reflect on your own main.tf and variables.tf files: propose one additional output value you would add, and explain what practical purpose it would serve for a teammate using your module.

Week 7: CloudFormation Concepts and Multi-Cloud Automation

Mapped CLO: CLO-1, CLO-3

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Explain the concept of provider-native declarative templating and how it compares to cloud-agnostic IaC tools.
- Identify the trade-offs between single-cloud and multi-cloud/cloud-agnostic automation strategies.
- Compare an open-source cloud-agnostic approach with a provider-native templating approach conceptually.
- Produce a structured comparison of two infrastructure automation approaches for a given scenario.

Hour 1 - Multi-Cloud Automation Concepts

Introduce the general concept of provider-native declarative infrastructure templating (of which AWS CloudFormation is the best-known commercial example) as JSON/YAML templates interpreted directly by a single cloud provider's own control plane, without a third-party tool in between.

Contrast this with cloud-agnostic tools (such as Terraform/OpenTofu) that use a common configuration language and a plugin (provider) model to target many different clouds, on-premises platforms, and SaaS services from one codebase.

Discuss the business drivers for multi-cloud strategy: avoiding vendor lock-in, meeting data-residency/regulatory requirements, and negotiating leverage - alongside the added complexity multi-cloud introduces (skills, tooling, cost management).

Introduce Crossplane (open source) as an example of a Kubernetes-native, cloud-agnostic infrastructure provisioning approach that extends the orchestration concepts from Week 5 to infrastructure provisioning itself.

Hour 2 - Conceptual Comparison of Approaches

Compare provider-native templating and cloud-agnostic IaC across dimensions: portability, learning curve, depth/timeliness of feature support for a given provider's newest services, and ecosystem/module reuse.

Discuss why cloud-agnostic tools are generally slightly behind on 'day-one' support for a provider's newest features (since a plugin must be updated), while provider-native tools usually support new features immediately.

Discuss the concept of a 'lowest common denominator' problem in true multi-cloud abstractions, and why many organizations instead standardize on one cloud-agnostic tool (Terraform/OpenTofu) while writing provider-specific modules rather than seeking one script that runs unmodified on every cloud.

Present a structured comparison framework (a table of criteria) that students will apply in the lab/assignment this week.

Hour 3 - Assignment: Comparative Analysis Exercise

Students are given a short, provider-native template example (JSON/YAML, presented for reading and analysis only, not for execution against a paid account) describing a small set of resources.

Students rewrite the equivalent infrastructure using Terraform/OpenTofu syntax (targeting the Docker or LocalStack provider used in Week 6, or written as a syntactically valid but non-applied example) to concretely compare the two authoring experiences.

Students complete a comparison table (portability, syntax verbosity, state management, multi-resource dependency handling) and write a short justified recommendation for a hypothetical company scenario provided by the instructor.

Groups present their comparison table and recommendation briefly, with peer questions, reinforcing CLO-3 evaluative skills ahead of the midterm review in Week 8.

```
# Illustrative provider-native style template (read-only comparison
exercise)
Resources:
  MyBucket:
    Type: "CloudProviderX::Storage::Bucket"
    Properties:
      BucketName: "course-demo-bucket"

# Equivalent cloud-agnostic style (Terraform/OpenTofu, illustrative)
resource "local_file" "bucket_placeholder" {
  filename = "./course-demo-bucket/.keep"
  content  = "placeholder for storage bucket concept"
}
```

Recommended Open-Source / Free Tools

- OpenTofu/Terraform (open source)
- Crossplane (open source, conceptual)
- LibreOffice Calc or Writer (free, for comparison table)

AI-Restricted Assessment Questions (10)

1. A company operates entirely within one cloud provider and has no near-term plans to change. Argue, using the trade-offs discussed in lecture, whether provider-native templating or a cloud-agnostic tool such as Terraform/OpenTofu would be the more pragmatic choice, and justify your reasoning.
2. Explain the 'lowest common denominator' problem in multi-cloud abstraction using your own comparison-table exercise from today's assignment as a concrete example.
3. A start-up wants to be 'multi-cloud ready' before they have even chosen a primary provider. Critically evaluate this goal, identifying at least one hidden cost this decision would impose from day one.
4. Using your rewritten Terraform/OpenTofu equivalent from today's exercise, identify one specific feature of the original provider-native template that was straightforward to reproduce, and one that required a meaningfully different approach.

5. Explain why cloud-agnostic tools are typically slower to support a cloud provider's newest features than that provider's own native templating service, using the plugin/provider model discussed in lecture.
6. Propose a specific scenario (industry, regulatory context, or technical constraint) in which true multi-cloud redundancy for a single workload would be justified despite its added complexity, and one where it would not.
7. Your comparison table from today's assignment includes a 'state management' row. Explain, referencing your own Week 6 Terraform state discussion, what the provider-native equivalent of 'state' is and how it differs conceptually.
8. Critically discuss why Crossplane's Kubernetes-native approach to provisioning connects the orchestration concepts of Week 5 with the infrastructure-provisioning concepts of Weeks 2 and 6, using your own words.
9. A manager asks you to 'just automate everything the same way regardless of which cloud we use.' Based on today's lecture, explain the practical limits of that instruction and propose a more realistic automation strategy.
10. Reflecting on your group's presentation today, identify one piece of peer feedback or question you received, and explain how it changed or reinforced your original recommendation.

Week 8: Midterm Review and Assessment

Mapped CLO: CLO-1, CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Consolidate and integrate concepts and skills covered in Weeks 1-7.
- Demonstrate comprehension of infrastructure automation principles (CLO-1).
- Demonstrate applied scripting and configuration-management competence (CLO-2).
- Complete a combined theory and lab-based midterm assessment.

Hour 1 - Conceptual Review (CLO-1 Focus)

Structured recap of Weeks 1, 2, and 7: definitions of infrastructure automation, the automation maturity model, IaC principles (declarative vs imperative, idempotency, drift), and multi-cloud/cloud-agnostic trade-offs.

Facilitate an open question-and-answer session addressing common areas of confusion identified from assignment and quiz performance in Weeks 1-7.

Provide a consolidated concept map linking terminology across weeks (provisioning, configuration management, orchestration) to reinforce how the topics build on one another.

Distribute a review guide summarizing key terms, and clarify the format and coverage of the upcoming midterm examination.

Hour 2 - Applied/Hands-On Review (CLO-2 Focus)

Guided walkthrough recap of the Week 3 scripting lab, Week 4 Ansible lab, Week 5 Kubernetes lab, and Week 6 Terraform/OpenTofu lab, focusing on the commands and workflows students will be expected to reproduce.

Run a timed practice exercise: students are given a small, previously unseen but similar task (e.g., writing a short playbook task or a Terraform variable block) under exam-like conditions, then review the model solution together.

Address common lab pitfalls observed across the semester so far: quoting errors in Bash, YAML indentation errors in Ansible/Kubernetes manifests, and state-file handling mistakes in Terraform/OpenTofu.

Open lab/office-hour style session for students to ask clarifying questions on any outstanding exercises from Weeks 1-7.

Hour 3 - Midterm Examination (Theory + Lab)

Administer the midterm examination consisting of a theory component (covering CLO-1 concepts: definitions, principles, comparisons) and a lab/practical component (covering CLO-2 skills: writing or correcting a short script, playbook, or IaC configuration).

Theory component format: short-answer and scenario-based questions requiring explanation and justification rather than pure recall, consistent with the assessment style used throughout the course.

Lab component format: a hands-on task performed in the same lab environment used throughout the semester (Bash/Python, Ansible with Vagrant, or Terraform/OpenTofu with the local/Docker provider), assessed for correctness and, where relevant, idempotency.

Debrief logistics: submission method, time allocation per component, and permitted reference materials as specified by the instructor.

Recommended Open-Source / Free Tools

- All open-source tools used in Weeks 1-7 (Git, Bash, Python, Ansible, Vagrant/VirtualBox, Docker, Kubernetes/Minikube, OpenTofu/Terraform)

AI-Restricted Assessment Questions (10)

1. Synthesis Weeks 1 through 7: construct a single narrative describing how a small company might progress from fully manual server administration to a mature, version-controlled, orchestrated infrastructure, naming which week's concept applies at each stage.
2. Explain how idempotency (Week 2/3), Ansible's module-based state checking (Week 4), and Kubernetes' reconciliation loop (Week 5) are all expressions of the same underlying principle, using your own words.
3. A previous employer of yours (real or hypothetical) manually configures every new server. Using concepts from at least three different weeks, design a phased automation adoption plan and justify the order of phases.
4. Critically compare the risk profile of a configuration error in a Bash script (Week 3), an Ansible playbook (Week 4), and a Terraform configuration (Week 6) being applied at scale, identifying which you consider highest-risk and why.
5. Explain why the course sequence introduced scripting (Week 3) before configuration management (Week 4), and configuration management before orchestration (Week 5), connecting this to the automation maturity model from Week 1.
6. During your own midterm lab task, describe one debugging step you took that referenced a concept from an earlier week (e.g., checking YAML indentation, or re-running a command to test idempotency).
7. A classmate argues Terraform/OpenTofu (Week 6) makes Ansible (Week 4) unnecessary. Evaluate this claim, identifying the distinct responsibilities of provisioning versus configuration management.
8. Explain, using the Week 7 comparison framework, how you would decide between a cloud-agnostic and provider-native approach for a company you designed in your own phased adoption plan above.
9. Reflect on your own performance across the Weeks 1-7 assignments and quizzes: identify one CLO-1 (conceptual) area and one CLO-2 (applied) area you found most challenging, and explain specifically why.
10. Propose one realistic addition to the automation toolchain covered so far (Weeks 1-7) that you believe would most improve reliability for a mid-sized company, and justify your choice using at least two course concepts.

Week 9: Automated Provisioning and Scaling

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Explain horizontal and vertical scaling and the triggers that initiate automated scaling.
- Describe how Kubernetes implements automated scaling through the Horizontal Pod Autoscaler and cluster auto-scaling concepts.
- Configure and test an auto scaling policy in a lab environment.
- Evaluate the trade-offs of automated scaling decisions under load.

Hour 1 - Scaling Concepts and Triggers

Define horizontal scaling (adding/removing instances of a resource, e.g., more pod replicas or more servers) versus vertical scaling (increasing/decreasing the resource allocation of an existing instance, e.g., more CPU/memory to one container).

Discuss common scaling triggers: CPU/memory utilization thresholds, request-queue length, custom application metrics, and scheduled/predictive scaling based on known traffic patterns.

Explain automated provisioning as the broader practice of automatically creating the underlying compute resources (nodes/VMs) needed to support scaling decisions, distinguishing pod-level scaling from cluster/node-level scaling.

Discuss the cost and reliability trade-offs of aggressive versus conservative scaling policies, including the risk of 'thrashing' (rapid scale up/down cycles) if thresholds are poorly tuned.

Hour 2 - Autoscaling in Kubernetes

Introduce the metrics-server (open source) component required for Kubernetes to expose resource-usage metrics to the auto-scaling subsystem.

Explain the Horizontal Pod Autoscaler (HPA): how it queries metrics, compares against target thresholds defined in its specification, and adjusts the replica count of a Deployment automatically, connecting directly to the Week 5 Deployment concepts.

Introduce, at a conceptual level, cluster auto-scaling (adding/removing worker nodes automatically based on unschedulable pods) and note that this is typically demonstrated conceptually in a local lab since it depends on a cloud provider's node-provisioning API.

Discuss how automated scaling connects back to Week 2's IaC principles: scaling policies themselves are usually defined declaratively (as part of the manifest) rather than triggered by imperative scripts.

Hour 3 - Lab: Configuring the Horizontal Pod Autoscaler

Enable the metrics-server add-on in Minikube (or install it in Kind) and confirm metrics are being reported using 'kubectl top pods'. Because the HPA calculates utilization as a percentage of each container's CPU request, first confirm that the Deployment declares one (the Week 5 manifest did not); for example, run 'kubectl set resources deployment web-app --requests=cpu=100m'. Without a CPU request the HPA reports <unknown> for its target.

Define an HPA resource targeting the Week 5 Deployment, specifying minimum and maximum replica counts and a target CPU utilization percentage.

Generate synthetic CPU load against the deployed application (e.g., using a small open-source load-generation tool or a simple busy-loop script) and observe, using 'kubectl get hpa -w', the replica count increasing automatically.

Stop the load generator and observe the scale-down behavior, discussing the default cool-down/stabilisation window Kubernetes applies to avoid thrashing.

```
# hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50

# Prerequisite: the HPA measures utilization against each container's
CPU *request*
# kubectl set resources deployment web-app --requests=cpu=100m

# Commands
# kubectl apply -f hpa.yaml
# kubectl get hpa -w
# kubectl top pods
```

Recommended Open-Source / Free Tools

- Kubernetes (open source)
- Minikube/Kind
- metrics-server (open source)
- hey or k6 (open source load-testing tools)

AI-Restricted Assessment Questions (10)

1. Explain, using your own hpa.yaml from today's lab, the exact sequence of events that occurs from a CPU spike to a new pod becoming available, naming each Kubernetes component involved.

2. Your HPA is configured with minReplicas: 2 and maxReplicas: 6. Critically discuss a scenario in which this range could be insufficient during a genuine traffic surge, and propose a specific adjustment with justification.
3. During today's lab, describe what you observed in the 'kubectl get hpa -w' output when you stopped the load generator, and explain why Kubernetes does not scale down instantly.
4. Compare horizontal and vertical scaling for a stateful database workload versus a stateless web front-end, explaining why one scaling strategy is typically preferred for each.
5. A poorly tuned autoscaling policy causes a deployment to oscillate between 2 and 6 replicas every few minutes under moderate, steady load. Diagnose the likely cause and propose a specific configuration change to fix it.
6. Explain why automated provisioning at the cluster/node level (not just the pod level) is necessary for genuine elasticity, referencing the distinction made in lecture between pod-level and node-level scaling.
7. Critically evaluate the statement: 'More aggressive autoscaling is always better because it maximizes responsiveness to load.' Identify at least one cost or reliability risk this ignores.
8. Your organization runs a predictable daily traffic pattern (high load 9am-5pm, low load overnight). Propose whether reactive metric-based autoscaling, scheduled scaling, or a combination would better fit this pattern, and justify your answer.
9. Explain how the declarative nature of the HPA resource (Week 2 principles) differs from writing a custom script that periodically checks CPU usage and manually issues 'kubectl scale' commands.
10. Reflect on your own lab session: report the specific averageUtilization value you used and explain, based on your load-test results, whether you would raise or lower it for a production-like workload, with justification.

Week 10: Monitoring and Logging in Automated Infrastructure

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Explain the three pillars of observability: metrics, logs, and traces.
- Describe Prometheus's architecture for metrics collection and alerting.
- Describe how Grafana is used to visualize metrics through dashboards.
- Deploy and configure a monitoring stack in a lab environment.

Hour 1 - Observability Fundamentals

Introduce the three pillars of observability: metrics (numeric time-series data, e.g., CPU usage), logs (discrete, timestamped event records), and traces (records of a request's path through distributed components).

Discuss why observability is treated as a core automation concern rather than an afterthought: automated systems (auto-scaling, self-healing) depend on accurate, timely metrics to make correct decisions, directly connecting to Week 9's auto-scaling triggers.

Introduce the concept of alerting: defining thresholds or conditions on metrics/logs that automatically notify a team or trigger a remediation workflow.

Discuss centralized logging as a necessity once infrastructure is distributed across many containers/nodes, since individual server log files are no longer sufficient for troubleshooting.

Hour 2 - Prometheus and Grafana

Explain Prometheus's pull-based architecture: the Prometheus server periodically scrapes metrics endpoints exposed by applications/exporters, storing them in its own time-series database.

Introduce PromQL at a conceptual level as the query language used to select and aggregate time-series data for dashboards and alerting rules.

Introduce Grafana as an open-source visualization layer that connects to Prometheus (and other data sources) to build dashboards, and discuss the difference between a dashboard (for human observation) and an alerting rule (for automated notification).

Discuss Alertmanager (part of the Prometheus ecosystem, open source) as the component responsible for routing, grouping, and silencing alerts generated from Prometheus rules.

Hour 3 - Lab: Deploying a Monitoring Stack

Install Helm (the open-source Kubernetes package manager) and use the community 'kube-prometheus-stack' Helm chart to deploy Prometheus, Alertmanager, and Grafana onto the Minikube/Kind cluster used in Weeks 5 and 9.

Access the Grafana UI (via port-forward) using the default open-source dashboard bundled with the chart, and explore pre-built dashboards showing cluster and pod resource usage.

Create one custom Grafana panel visualizing CPU usage for the Week 5/9 web-app deployment, connecting the visualization directly to the auto-scaling behavior observed last week.

Define a simple Prometheus alerting rule (e.g., alert if a pod's CPU usage exceeds a threshold for more than two minutes) and confirm it appears correctly in the Alertmanager UI.

```
# Install monitoring stack via Helm (open source)
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack

# Access Grafana locally
kubectl port-forward svc/monitoring-grafana 3000:80

# Example Prometheus alert rule (YAML). With kube-prometheus-stack,
place it inside a PrometheusRule resource (spec.groups).
groups:
  - name: web-app-alerts
    rules:
      - alert: HighCPUUsage
        expr: sum by (pod)
(rate(container_cpu_usage_seconds_total{pod=~"web-app.*"}[2m])) > 0.8
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "High CPU usage detected on web-app pod"
```

Recommended Open-Source / Free Tools

- Prometheus (open source)
- Grafana (open source)
- Alertmanager (open source)
- Helm (open source)

AI-Restricted Assessment Questions (10)

1. Explain, using today's lab deployment, why Prometheus's pull-based model (server scrapes targets) differs from a push-based logging pipeline, and identify one advantage of the pull model for a dynamically scaling environment such as the one from Week 9.
2. Your custom Grafana panel from today's lab visualizes CPU usage for the web-app deployment. Explain how this dashboard could have helped you diagnose the autoscaling thrashing scenario discussed in Week 9, even before configuring an alert.
3. Compare metrics, logs, and traces using a concrete troubleshooting scenario (e.g., a single user request that failed): explain what each of the three pillars would tell you that the others would not.
4. Critically evaluate the statement: 'If we have dashboards, we do not need automated alerting.' Identify the specific operational gap this statement overlooks.

5. Your Prometheus alerting rule from today's lab uses 'for: 2m' before firing. Explain the purpose of this delay and describe a specific scenario where omitting it would cause a problematic number of false alerts.
6. Explain why centralized logging becomes operationally necessary once an application is orchestrated across multiple pods/nodes (Week 5), rather than relying on 'kubectl logs' against individual pods one at a time.
7. A dashboard shows CPU usage steadily rising over several days with no corresponding increase in traffic. Propose two specific hypotheses this observation might support, and explain what additional data (metrics, logs, or traces) you would gather to test each.
8. Explain the relationship between Prometheus, Alertmanager, and Grafana in the stack you deployed today, describing specifically which component is responsible for detection, which for routing/notification, and which for visualization.
9. Design (in prose) one additional alerting rule you would add for the web-app deployment beyond CPU usage, and justify why the metric you chose is operationally meaningful.
10. Reflect on your own Helm-based deployment today: describe one issue you encountered (e.g., pod not ready, port-forward failure) and the specific steps you took to resolve it.

Week 11: Integration with CI/CD Pipelines

Mapped CLO: CLO-2

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Explain the stages of a CI/CD pipeline and their relationship to infrastructure automation.
- Describe Jenkins' pipeline-as-code model and GitHub Actions' workflow model.
- Integrate infrastructure automation (IaC/configuration management) commands into an automated pipeline.
- Evaluate pipeline design choices for safety and reliability.

Hour 1 - CI/CD Concepts

Define Continuous Integration (frequent merging and automated testing of code changes) and Continuous Delivery/Deployment (automated, reliable release of validated changes to an environment), distinguishing delivery (a human approves the final release step) from deployment (fully automatic).

Walk through typical pipeline stages: source checkout, build, automated test, artifact/image creation, and deploy - and map each stage to the tools introduced earlier in the course (e.g., the deploy stage may run Ansible or Terraform/OpenTofu commands).

Discuss why infrastructure changes (not just application code) benefit from being run through a CI/CD pipeline: automated 'plan' review, mandatory peer approval before 'apply', and a consistent, auditable execution environment rather than a human running commands from a personal laptop.

Introduce the concept of pipeline as code: defining the pipeline itself in a version-controlled file alongside the project, rather than through a UI-configured job.

Hour 2 - Jenkins and GitHub Actions

Introduce Jenkins (open source) architecture: controller and agent nodes, and the Jenkinsfile (Groovy-based pipeline-as-code definition) describing stages and steps.

Introduce GitHub Actions (free for public repositories and generous free-tier minutes for private ones) and its YAML-based workflow files, triggered by repository events such as pushes or pull requests.

Compare the two: Jenkins as a self-hosted, highly extensible option requiring server maintenance, versus GitHub Actions as a hosted, tightly Git-integrated option with less operational overhead for smaller teams.

Discuss secrets management within pipelines (e.g., GitHub Actions encrypted secrets, or Jenkins Credentials plugin) as a security practice that anticipates the Week 12 topic.

Hour 3 - Lab: Building an Automation Pipeline

Students create a GitHub Actions workflow file in their course repository that automatically runs on every push: checking out the repository, and running 'terraform fmt -check' and 'terraform plan' (or 'tofu' equivalents) against the Week 6 configuration using the Docker/local provider.

Extend the workflow to run an Ansible playbook's syntax check ('ansible-playbook --syntax-check') against the Week 4 playbook as an additional automated validation stage.

Introduce manual-approval concepts by discussing (and where feasible configuring) a required review/approval step before an 'apply' stage would run, even though the course lab environment will stop at 'plan' for safety.

As an optional stretch task for students with prior exposure, briefly demonstrate an equivalent Jenkinsfile achieving the same checkout-plan validation pipeline, for architectural comparison.

```
# .github/workflows/iac-validate.yml
name: IaC Validation Pipeline
on: [push]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup OpenTofu
        uses: opentofu/setup-opentofu@v1
      - name: Format Check
        run: tofu fmt -check
      - name: Init
        run: tofu init
      - name: Plan
        run: tofu plan
      - name: Install Ansible
        run: pipx install ansible-core
      - name: Ansible Syntax Check
        run: ansible-playbook --syntax-check playbook.yml
```

Recommended Open-Source / Free Tools

- GitHub Actions (free tier)
- Jenkins (open source, optional comparison)
- OpenTofu/Terraform
- Ansible

AI-Restricted Assessment Questions (10)

1. Explain, using your own iac-validate.yml workflow from today's lab, why running 'terraform plan' automatically on every push is safer than each engineer running 'plan'/'apply' manually from their own laptop.
2. Distinguish Continuous Delivery from Continuous Deployment using a concrete example relevant to infrastructure changes, and justify which model you would recommend for changes to a production database configuration, with reasoning.
3. Your GitHub Actions workflow currently stops at the 'plan' stage rather than automatically applying. Critically discuss the trade-off between fully automating the 'apply' step versus requiring a manual approval gate.

4. Compare Jenkins and GitHub Actions using the architectural differences discussed in lecture, and justify which you would recommend for a five-person start-up with no dedicated infrastructure/DevOps engineer.
5. Explain why 'pipeline as code' (a Jenkinsfile or GitHub Actions YAML file stored in the repository) is preferable to configuring the same pipeline steps manually through a web UI, connecting your answer to the Week 2 IaC principles.
6. During today's lab, your Ansible syntax-check step could fail even though the playbook worked correctly when you ran it manually in Week 4's lab. Propose two specific reasons this discrepancy could occur in a pipeline environment.
7. A pipeline is configured to automatically run 'terraform apply' immediately after every merge to the main branch with no approval step. Identify one specific incident scenario this could cause, and propose a concrete pipeline change to mitigate it.
8. Explain how secrets management (briefly introduced this week) connects to the security topics you will study in Week 12, using a specific example of a secret your own workflow file would need if it were provisioning real cloud resources.
9. Critically evaluate: 'Once a CI/CD pipeline exists, manual infrastructure changes should never be necessary again.' Identify at least one legitimate exception to this claim.
10. Reflect on your own workflow file from today's lab: describe one stage you would add before this pipeline could be considered ready to manage a real team's shared infrastructure, and justify your choice.

Week 12: Security and Compliance in Automation

Mapped CLO: CLO-3

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Explain core security principles relevant to automated infrastructure: secrets management, least privilege, and policy as code.
- Identify open-source tools for compliance scanning and static analysis of infrastructure code.
- Evaluate a system's or codebase's security/compliance posture using automated scanning tools.
- Design mitigation for identified security or compliance findings.

Hour 1 - Security Principles in Automation

Discuss why automation both improves and complicates security: consistent, auditable configuration reduces certain risks, but a single flawed script or template can now propagate a vulnerability across an entire fleet instantly.

Introduce the principle of least privilege as applied to automation credentials: automation tools (Ansible, Terraform/OpenTofu, CI/CD pipelines) should hold only the minimum permissions necessary for their specific task, not broad administrative access.

Discuss secrets management: why credentials, API keys, and certificates must never be hard-coded into scripts, playbooks, or IaC files, and introduce open-source secret-handling approaches (e.g., environment variables sourced from a secrets manager, or HashiCorp Vault's open-source edition).

Introduce policy as code: expressing security and compliance rules themselves as version-controlled, automatically enforced code (e.g., 'no storage resource may be created without encryption enabled'), extending the IaC principles from Week 2 into the governance domain.

Hour 2 - Open-Source Compliance and Scanning Tools

Introduce OpenSCAP (open source) for automated compliance scanning of operating systems against established security baselines (e.g., CIS benchmarks).

Introduce Lynis (open source) as a lightweight host-based security auditing tool suitable for quick, scriptable checks within a CI pipeline or as a standalone audit.

Introduce static analysis for IaC code specifically: tools such as tfsec and Checkov (both open source) that scan Terraform/OpenTofu configurations for insecure defaults (e.g., an open storage bucket, an unrestricted network rule) before the code is ever applied.

Discuss integrating these scanners into the Week 11 CI/CD pipeline as an additional automated gate, so that insecure configurations are flagged automatically rather than relying on manual code review alone.

Hour 3 - Lab: Auditing and Scanning

Run Lynis against the Vagrant VM configured in Week 4's Ansible lab, review the generated hardening index and findings, and identify the top three recommended remediations.

Run Checkov or tfsec against the Terraform/OpenTofu configuration written in Week 6, review the findings report, and correct at least two flagged issues (e.g., adding an explicit security-relevant argument that was previously left at an insecure default).

Add the static-analysis scan as an additional step in the Week 11 GitHub Actions workflow, so that future pushes are automatically checked for security misconfigurations.

Discuss, as a group, which findings from the scans represent genuine risk versus acceptable trade-offs for a lab/teaching environment, reinforcing the evaluative (CLO-3) skill of contextualised risk assessment.

```
# Lynis system audit (open source)
sudo lynis audit system

# Checkov scan against Terraform/OpenTofu code (open source)
pip install checkov
checkov -d ./week6-lab

# Add to GitHub Actions workflow (.github/workflows/iac-validate.yml)
      - name: Security Scan with Checkov
        run: pipx run checkov -d .
```

Recommended Open-Source / Free Tools

- Lynis (open source)
- OpenSCAP (open source)
- Checkov (open source)
- tfsec (open source)

AI-Restricted Assessment Questions (10)

1. Explain, using your own Lynis scan output from today's lab, what the hardening index represents and identify the single finding you judged to be the most operationally significant, with justification.
2. Your Checkov/tfsec scan flagged a resource in your Week 6 configuration as insecure by default. Describe the specific finding, the change you made to remediate it, and how you confirmed the fix using a re-scan.
3. Explain the principle of least privilege as it applies specifically to the credentials your Week 11 GitHub Actions pipeline would need if it were provisioning real cloud resources, rather than the local/Docker provider used in this course.
4. Critically discuss why hard-coding a password or API key directly into an Ansible playbook (Week 4) or Terraform variable file (Week 6) is a security risk even in a private Git repository, referencing specific consequences.
5. Design (in prose) a policy-as-code rule you would add to your Week 11 pipeline to automatically block any Terraform/OpenTofu resource lacking a required tag or setting, and explain how this extends the IaC principles from Week 2 into governance.

6. Compare OpenSCAP-style compliance benchmarking with Checkov/tfsec-style IaC static analysis: explain what stage of the automation life-cycle each is best suited to, using the CI/CD pipeline stages from Week 11 as reference points.
7. A team argues that adding security scanning to their CI/CD pipeline (Week 11) will 'slow down deployment too much.' Evaluate this argument and propose a specific way to balance scan thoroughness with pipeline speed.
8. Explain why automated infrastructure can propagate a single security misconfiguration far faster than manual administration, using a specific example from your own Week 6 or Week 5 lab work.
9. Reflect on the group discussion in today's lab: describe one scan finding your group classified as an acceptable trade-off for a teaching/lab environment, and explain specifically why the same finding would not be acceptable in a production system.
10. Propose one additional open-source security or compliance tool (beyond those covered this week) you would investigate for your own future automation projects, and justify what gap it would address.

Week 13: Enterprise Automation Case Studies

Mapped CLO: CLO-3

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Analyze real-world (anonymized/generalized) narratives of enterprise infrastructure automation adoption.
- Identify the business and technical drivers behind large-scale automation transformations.
- Evaluate the trade-offs and risks encountered during enterprise automation initiatives.
- Present a structured group analysis of an automation case study.

Hour 1 - Case Study Methodology and Presentation

Introduce a structured framework for analyzing an automation case study: initial state/problem, drivers for change, tools and practices adopted, implementation challenges, measurable outcomes, and lessons learned.

Present two or three generalized enterprise automation narratives (drawn from publicly discussed industry patterns rather than any single proprietary account) covering different starting points: a legacy on-premises data center modernization, a start-up scaling rapidly on cloud infrastructure, and a regulated-industry compliance-driven automation adoption.

For each narrative, connect the tools and practices described back to specific weeks of this course (e.g., configuration management adoption to Week 4, orchestration adoption to Week 5, CI/CD integration to Week 11, security/compliance automation to Week 12).

Discuss common failure patterns in enterprise automation initiatives: automating a broken process without first fixing it, insufficient training leading to shadow manual processes, and underestimating the cultural change required.

Hour 2 - Group Analysis Workshop

Divide students into small groups, each assigned one of the case-study narratives (or a case study the instructor provides in more detail) for structured analysis using the framework introduced in Hour 1.

Groups identify which automation maturity stage (Week 1) the organization started at and which stage it reached, and map specific tools/practices from the course onto each stage of the transformation.

Groups critically evaluate at least one decision in the case study they might have made differently, and justify an alternative approach using course concepts (e.g., a different configuration-management tool, a different scaling strategy, or a different security-scanning approach).

Groups prepare a concise presentation (structured around the case-study framework) summarizing their analysis, findings, and one key recommendation for the organization in their case study.

Hour 3 - Group Presentations and Discussion

Each group presents their case-study analysis (approximately 8-10 minutes) covering the initial state, drivers, tools adopted, challenges, outcomes, and their own critical evaluation/recommendation.

Facilitate structured peer questioning after each presentation, encouraging groups to challenge each other's assumptions and proposed alternatives using specific course concepts.

Instructor provides feedback connecting each group's analysis explicitly to CLO-3 evaluative skills: assessing trade-offs, and aligning automation strategy with organizational context.

Close with a synthesis discussion: what common threads emerged across the different case studies, and how these threads connect to the best-practices topic scheduled for Week 14.

Recommended Open-Source / Free Tools

- LibreOffice Impress / Only Office (free, for presentations)
- Git/GitHub (for shared group notes, optional)

AI-Restricted Assessment Questions (10)

1. Using the case-study framework from today's lecture, identify the automation maturity stage (Week 1) your assigned organization started at, and provide two specific pieces of evidence from the narrative supporting your classification.
2. Your group proposed one alternative decision the organization in your case study could have made. Justify this alternative using at least two specific course concepts (naming the relevant week/tool) rather than general opinion.
3. Explain how the security and compliance automation practices from Week 12 would apply differently to a regulated-industry organization compared to a fast-growing start-up, using the case studies discussed today as reference points.
4. Critically evaluate the failure pattern 'automating a broken process without first fixing it' using a specific, plausible example relevant to one of the case studies discussed, and propose how a phased approach (Week 1 maturity model) could have avoided it.
5. Compare the CI/CD integration challenges (Week 11) likely faced by a legacy on-premises organization migrating to automation versus a cloud-native start-up, and explain why the same tool (e.g., Jenkins or GitHub Actions) might be adopted differently by each.
6. During today's peer-question session, describe one question or challenge your group received, and explain how you responded, or how your analysis would change in light of that challenge.
7. Explain why 'insufficient training leading to shadow manual processes' is a risk specifically for configuration-management and orchestration adoption (Weeks 4-5), rather than for scripting alone (Week 3).
8. Propose one measurable outcome (a specific metric) an organization could track to evaluate whether an infrastructure automation initiative like the ones discussed today is succeeding, and justify why that metric is meaningful.
9. Critically discuss whether the tools and practices covered in this course (all open source/free) would need to change for an enterprise with strict vendor-support and legal-compliance requirements, using specific examples from today's case studies.
10. Reflect on the synthesis discussion at the end of class: identify one common thread across the different case studies presented today, and explain how it foreshadows the best-practices topic for Week 14.

Week 14: Best Practices in Infrastructure Automation

Mapped CLO: CLO-3

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Identify best practices for writing modular, testable, and well-documented IaC.
- Explain GitOps principles and their relationship to automated infrastructure delivery.
- Synthesize best practices across scripting, configuration management, orchestration, and security into a coherent approach.
- Produce a written set of automation best-practice guidelines for a given scenario.

Hour 1 - IaC and Code-Quality Best Practices

Consolidate best practices for IaC (Weeks 2, 6): modularity (small, reusable modules with clear inputs/outputs), consistent naming conventions, and thorough documentation (README files, inline comments explaining non-obvious decisions).

Discuss testing strategies for infrastructure code: linting (e.g., 'terraform fmt'/'validate', 'ansible-lint', both open source), plan-review as a form of testing (Week 6/11), and where feasible, automated integration tests that verify a deployed resource behaves as expected.

Discuss version-pinning best practices: pinning provider, module, and tool versions explicitly to avoid unexpected behavior changes from upstream updates, connecting to the idempotency and repeatability principles from Week 2.

Discuss code review practices for infrastructure changes: requiring peer review of IaC pull requests just as with application code, reinforcing the CI/CD pipeline gates introduced in Week 11.

Hour 2 - GitOps and Organizational Best Practices

Introduce GitOps as an operational model where Git is the single source of truth for both application and infrastructure desired state, and an automated agent continuously reconciles the live environment to match the repository (extending the Week 5 Kubernetes reconciliation-loop concept and the Week 11 pipeline concept).

Introduce open-source GitOps tools conceptually (e.g., Argo CD, Flux) as agents that watch a Git repository and automatically apply changes to a Kubernetes cluster, contrasting this 'pull-based' deployment model with the 'push-based' pipeline model used in Week 11's lab.

Discuss organizational best practices: clear ownership of automation code, a documented on-call/incident process for when automation misbehaves, and a culture that treats automation code with the same rigor as production application code.

Discuss the balance between standardization (consistent tools/patterns across teams) and flexibility (allowing teams to choose appropriate tools for their specific workload), referencing the Week 7 multi-cloud/tooling trade-off discussion.

Hour 3 - Assignment: Writing Automation Best-Practice Guidelines

Students are given a short scenario describing a mid-sized organization with multiple teams, inconsistent tool usage, and a recent incident caused by an un-reviewed manual infrastructure change.

Students individually (or in pairs) draft a concise best-practices document for this organization, covering: version control and code review policy, testing/linting requirements, secrets-management policy (Week 12), CI/CD gating requirements (Week 11), and a recommendation on GitOps adoption where relevant.

Students must justify each recommended practice by referencing the specific risk or incident type it addresses, rather than listing generic advice, reinforcing CLO-3 evaluative and design skills.

Submitted documents are peer-reviewed in pairs against a short rubric (clarity, specificity, and justification) before final submission to the instructor.

Recommended Open-Source / Free Tools

- Git/GitHub
- ansible-lint (open source)
- terraform fmt/validate (open source)
- Argo CD or Flux (open source, conceptual)

AI-Restricted Assessment Questions (10)

1. Using the incident scenario from today's assignment (an un-reviewed manual change), design one specific best-practice rule that would have prevented it, and justify why it directly addresses the root cause described.
2. Explain GitOps using the reconciliation-loop concept from Week 5's Kubernetes lecture: describe, step by step, what would happen if someone manually changed a live Kubernetes resource that a GitOps agent was also managing.
3. Compare the 'push-based' pipeline model from Week 11 with the 'pull-based' GitOps model discussed today, identifying one specific advantage and one specific disadvantage of each for a security-conscious organization (Week 12).
4. Your best-practices document from today's assignment includes a version-pinning recommendation. Explain, using the idempotency and repeatability principles from Week 2, why failing to pin provider/module versions can silently break a previously working IaC configuration.
5. Critically evaluate the claim: 'Standardizing every team on exactly the same tools is always the best practice for a growing organization.' Using the Week 7 multi-cloud trade-off discussion, identify a scenario where deliberate flexibility would be preferable.
6. Explain why code review for infrastructure changes (this week) is treated as equally important as code review for application code, connecting your answer to at least one specific risk discussed in Weeks 6 or 12.
7. Design one specific linting or testing rule (e.g., using ansible-lint or terraform validate) you would make mandatory in a CI/CD pipeline (Week 11) for a team new to infrastructure automation, and justify your choice.

8. During today's peer-review of best-practice documents, describe one piece of feedback you received (or gave), and explain how it improved the specificity or justification of the recommendation in question.
9. Propose a documented on-call/incident process for when an automated system (e.g., a GitOps agent or an autoscaler from Week 9) behaves unexpectedly, and justify why this process is itself a form of automation best practice.
10. Reflecting on Weeks 1 through 14 as a whole, identify the single best-practice you consider most foundational to reliable infrastructure automation, and justify your choice using evidence or reasoning from at least two different weeks.

Week 15: Capstone Lab - Design an Infrastructure as Code Solution

Mapped CLO: CLO-3

Duration: 3 Hours (2 Theory Hours + 1 Lab/Practical Hour)

Learning Objectives

- Integrate Terraform/OpenTofu and Ansible into a single combined provisioning-and-configuration solution.
- Design an end-to-end infrastructure automation solution for a defined scenario.
- Apply best practices, security considerations, and CI/CD integration learned throughout the semester.
- Produce a working capstone project artifact for evaluation.

Hour 1 - Capstone Briefing and Requirements

Present the capstone project brief: design and implement an Infrastructure as Code solution for a small web application, combining Terraform/OpenTofu (for provisioning, Weeks 2 and 6) and Ansible (for configuration management, Week 4) targeting the open-source/local lab environment used throughout the course (Docker/LocalStack provider, Vagrant, or Minikube as appropriate to the students' chosen architecture).

Present the grading rubric, mapped explicitly to CLO-3: correctness and functioning of the provisioned infrastructure, appropriate use of modules/idempotent design (Week 2/6), integration of at least one best practice from Week 14 (e.g., linting, version pinning, or documentation), and a brief written justification of design decisions.

Clarify project scope boundaries and permitted tool combinations, ensuring only open-source/free tools are used consistent with the course's tooling policy throughout.

Address logistics: submission format (a Git repository), deadline, and the format of the short design-justification write-up required alongside the working code.

Hour 2 - Architecture Design Session

Guide students (individually or in small teams) through a structured design session: sketching the target architecture (e.g., provisioned containers/VMs, network relationships, and the services/configuration each will run), before writing any code.

Facilitate discussion of design trade-offs specific to each student's chosen scenario: how to split responsibilities between Terraform/OpenTofu (what to provision) and Ansible (how to configure what has been provisioned), reinforcing the Week 4/6 boundary between provisioning and configuration management.

Encourage students to plan for at least one best-practice element from Week 14 (module structure, a README, or a lint/validate step) and one security consideration from Week 12 (e.g., no hard-coded secrets) as part of their design, not as an afterthought.

Instructor circulates to review each student's/team's architecture sketch and design plan before implementation begins, providing early feedback to prevent large rework later.

Hour 3 - Guided Implementation and Consultation

Dedicated implementation time in the lab environment: students write and test their combined Terraform/OpenTofu provisioning code and Ansible configuration playbooks against their approved design from Hour 2.

Instructor provides one-on-one/team consultation during implementation, focused on troubleshooting integration issues between the provisioning and configuration-management stages (e.g., ensuring Ansible's inventory correctly reflects resources just provisioned by Terraform/OpenTofu).

Students perform an initial end-to-end test of their solution (provision, then configure, then verify the application/service is reachable), and begin drafting the short design-justification write-up required for submission.

Wrap up with a clear statement of remaining work expected before the final project submission (aligned with Week 16's final review/exam), and reminders of the CLO-3 rubric criteria.

```
# Illustrative combined-solution skeleton
# Step 1: Terraform/OpenTofu provisions containers (Week 6 pattern)
resource "docker_container" "app_vm" {
  name  = "capstone-app"
  image = docker_image.base.image_id
}

output "app_container_name" {
  value = docker_container.app_vm.name
}

# Step 2: Generated/updated Ansible inventory reflects provisioned
resource
# inventory.ini
[app]
capstone-app ansible_connection=docker

# Step 3: Ansible playbook configures the provisioned container (Week
4 pattern)
- hosts: app
  tasks:
    - name: Install application dependencies
      apt:
        name: nginx
        state: present
```

Recommended Open-Source / Free Tools

- OpenTofu/Terraform (open source)
- Ansible (open source)
- Docker or Vagrant/VirtualBox
- Git/GitHub

AI-Restricted Assessment Questions (10)

1. Justify, for your own capstone design, the specific boundary you chose between what Terraform/OpenTofu provisions and what Ansible configures, referencing the Week 4/6 provisioning-versus-configuration-management distinction.
2. Describe one integration challenge you encountered (or anticipate) in ensuring your Ansible inventory correctly reflects resources just created by Terraform/OpenTofu, and propose a specific solution.
3. Explain how your capstone design incorporates at least one best practice from Week 14 (e.g., modularity, linting, or documentation), and justify why you selected that particular practice for this project.
4. Explain how your capstone design avoids hard-coded secrets, referencing the Week 12 secrets-management principles, even in a local/lab environment with no real credentials at stake.
5. Critically evaluate your own architecture sketch from Hour 2: identify one design decision you changed (or would change) after instructor feedback, and justify the revision.
6. Explain why testing your solution end-to-end (provision, then configure, then verify) is a stronger validation approach than testing the Terraform/OpenTofu and Ansible components in isolation, referencing the CI/CD integration concepts from Week 11.
7. Propose one way you would extend your capstone project with automated scaling (Week 9) or monitoring (Week 10) if given additional time, and justify why that extension would add the most value.
8. Your rubric requires a written design-justification. Draft (in a few sentences) the core justification for your single most important design decision, explaining which course concept most directly informed it.
9. Identify one security or compliance scan (Week 12, e.g., Checkov/tfsec or Lynis) you would run against your capstone code before considering it complete, and explain what specific risk it would help you rule out.
10. Reflecting on the entire course (Weeks 1-15), explain how your capstone project demonstrates the progression from basic scripting (Week 3) through configuration management (Week 4), provisioning (Week 6), and integration/best practices (Weeks 11-14) into a single coherent solution.